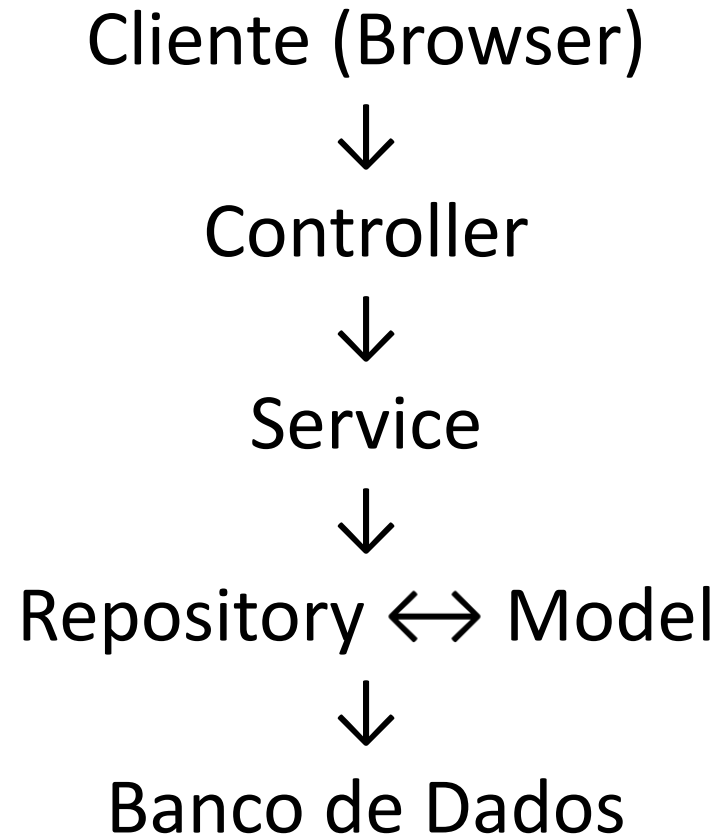


Arquitetura em Camadas (Flask)



Arquivo principal – app.py

```
1  from flask import Flask
2  from controller.cliente_controller import cliente_bp
3
4  app = Flask(__name__)
5
6  # registrar rotas
7  app.register_blueprint(cliente_bp)
8
9  if __name__ == "__main__":
10     app.run(debug=True)
```

Controller

- Recebe requisições HTTP
- Define rotas
- Retorna JSON

Controller – arquivo: cliente_controller.py

```
1  from flask import Blueprint, jsonify
2  from service.cliente_service import ClienteService
3
4  cliente_bp = Blueprint('cliente', __name__)
5
6  service = ClienteService()
7
8  # GET /clientes
9  @cliente_bp.route('/clientes', methods=['GET'])
10 def listar_clientes():
11     clientes = service.listar_clientes()
12     return jsonify(clientes)
13
14 # GET /clientes/1
15 @cliente_bp.route('/clientes/<int:id>', methods=['GET'])
16 def buscar_cliente(id):
17     cliente = service.buscar_cliente(id)
18
19     if cliente:
20         return jsonify(cliente)
21
22     return jsonify({"erro": "Cliente não encontrado"}), 404
```

O que é Blueprint ?

O Blueprint é um módulo de rotas que pode ser:

- registrado no app principal
- reutilizado
- organizado por funcionalidade

Comparação: Sem Blueprint vs Com Blueprint

Sem Blueprint:

```
# app.py (caótico )
@app.route('/clientes')
@app.route('/produtos')
@app.route('/pedidos')
@app.route('/login')
@app.route('/relatorios')
```

Tudo misturado
Difícil manter
Difícil trabalhar em equipe

Com Blueprint

Você divide por domínio:

```
controller/
  cliente_controller.py
  produto_controller.py
  pedido_controller.py
```

Cada arquivo = um módulo independente

Comparação: Sem Blueprint vs Com Blueprint

Critério	❌ Sem Blueprint	✅ Com Blueprint
Organização	Tudo no <code>app.py</code>	Separado por módulos
Estrutura	Código centralizado	Código distribuído
Exemplo de rotas	<code>/clientes</code> , <code>/produtos</code> , <code>/pedidos</code> , <code>/login</code> no mesmo arquivo	Cada rota em seu controller específico
Manutenção	Difícil 😞	Fácil 👍
Escalabilidade	Baixa	Alta 🚀
Trabalho em equipe	Complicado (conflitos no m  arquivo)	Facilitado (cada dev em um módulo)
Clareza do código	Confuso	Organizado
Reutilização	Difícil	Possível

Service

- Regra de negócio
- Processa dados
- Valida informações

Service – arquivo: cliente_service.py

```
1  from repository.cliente_repository import ClienteRepository
2
3  class ClienteService:
4
5      def __init__(self):
6          self.repository = ClienteRepository()
7
8      def listar_clientes(self):
9          clientes = self.repository.listar_todos()
10         return [c.to_dict() for c in clientes]
11
12     def buscar_cliente(self, id):
13         cliente = self.repository.buscar_por_id(id)
14         if cliente:
15             return cliente.to_dict()
16         return None
17
```

Repository

- Acesso ao banco de dados
- Executa SQL

Repository

archivo:

cliente_repository.py

```
1  from config.database import conectar
2  from models.cliente import Cliente
3  class ClienteRepository:
4
5      def listar_todos(self):
6          conexaso = conectar()
7          cursor = conexaso.cursor()
8
9          cursor.execute("SELECT * FROM clientes")
10         resultados = cursor.fetchall()
11
12         clientes = []
13         for row in resultados:
14             clientes.append(Cliente(row[0], row[1], row[2]))
15
16         cursor.close()
17         conexaso.close()
18
19         return clientes
20
21     def buscar_por_id(self, id):
22         conexaso = conectar()
23         cursor = conexaso.cursor()
24
25         cursor.execute("SELECT * FROM clientes WHERE id=%s", (id,))
26         resultado = cursor.fetchone()
27
28         cursor.close()
29         conexaso.close()
30
31         if resultado:
32             return Cliente(resultado[0], resultado[1], resultado[2])
33         return None
34
```

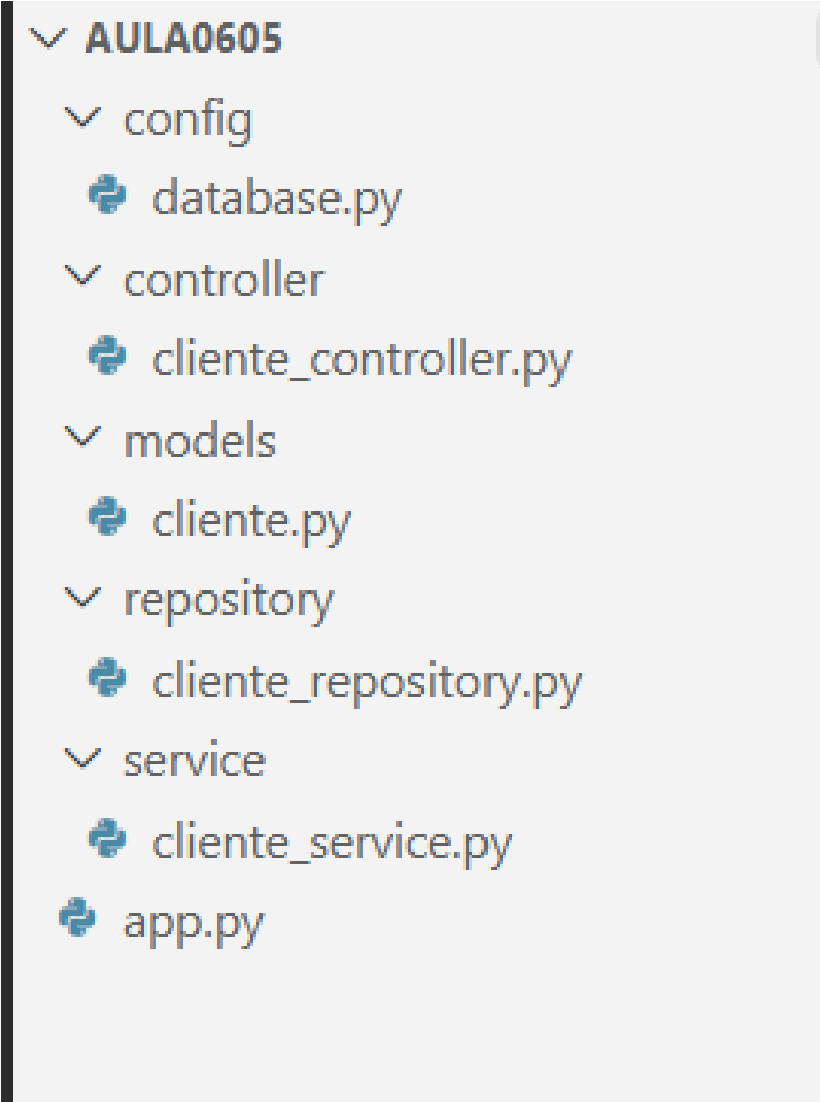
Model

- Representa dados
- Classe Cliente

Model – arquivo: cliente.py

```
1  class Cliente:
2      def __init__(self, id, nome, email):
3          self.id = id
4          self.nome = nome
5          self.email = email
6
7      def to_dict(self):
8          return {
9              "id": self.id,
10             "nome": self.nome,
11             "email": self.email
12         }
13
```

O projeto da aula0605



```

▼ AULA0605
  ▼ config
    database.py
  ▼ controller
    cliente_controller.py
  ▼ models
    cliente.py
  ▼ repository
    cliente_repository.py
  ▼ service
    cliente_service.py
  app.py

```

The image shows a file explorer window with a tree view of a project named 'AULA0605'. The project is expanded, showing several subdirectories and files. Each subdirectory is preceded by a downward-pointing chevron (▼), and each file is preceded by a blue icon representing a Python file. The subdirectories are 'config', 'controller', 'models', 'repository', and 'service'. The files are 'database.py' (under config), 'cliente_controller.py' (under controller), 'cliente.py' (under models), 'cliente_repository.py' (under repository), 'cliente_service.py' (under service), and 'app.py' (at the root of the project).

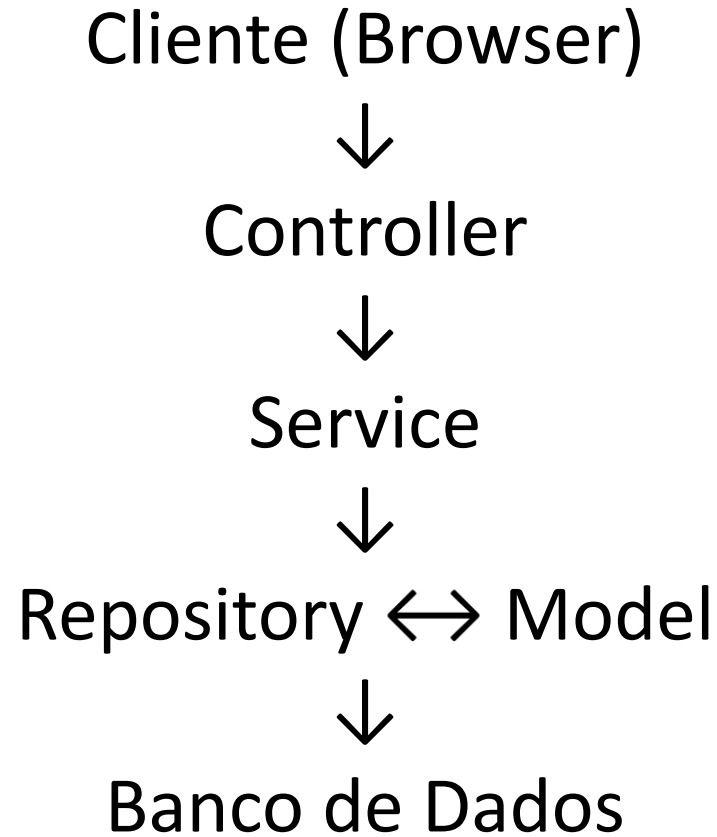
Conexão

- Responsável pela conexão com o banco

Conexão – arquivo: database.py

```
1  import mysql.connector
2
3  def conectar():
4      return mysql.connector.connect(
5          host="localhost",
6          user="root",
7          password="root",
8          database="aula_python",
9          port=3306
10     )
```


Fluxo Completo



Resumo

- Controller: recebe
- Service: processa
- Repository: banco
- Model: dados
- Conexão: acesso